



Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	7

Document Revision History

Version	Date	Author	Comments
1.0	3/22/26	Elijah Bastien	Filling out Executive Summary, Design Constraints, Domain Model
2.0	4/3/26		Evaluation
3.0	4/17/26		Recommendations

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Draw It or Lose It is a modern, web based reimagining of the classic 1980s televised drawing games. The project goal is to transition this concept into a distributed digital environment where teams compete to identify stock drawings as they render in real time. To meet the client's needs, our proposed solution focuses on a centralized game management system that ensures data integrity and competitive fairness. We will implement an architectural pattern to handle unique identifiers for games, teams, and players, preventing naming conflicts and ensuring that only one game instance exists in memory at any given time. The development will prioritize the Singleton Pattern and Unique Identification (UUID) logic. This ensures that the game engine remains stable and prevents phantom sessions or duplicate data that could crash the server as the user base scales. Hardware will remain on hold until the software architecture is finished to ensure the servers can handle the real time rendering demands.

Requirements

< Please note: While this section is not being assessed, it will support your outline of the design constraints below. *In your summary, identify each of the client's business and technical requirements in a clear and concise manner.*>

Design Constraints

The application must sync game states (timers, drawing progress, and scores) across multiple geographically dispersed players simultaneously. This means we must implement a real-time communication protocol (like WebSockets). Traditional HTTP requests will be too slow, if the drawing renders at 30 seconds for one player but 32 seconds for another, it creates an unfair competitive advantage. Another constraint is that only one instance of the game may exist in memory at any given time, and all game/team names must be globally unique. This requires a Singleton Design Pattern at the application level and a centralized database check during the "Create Team" phase. This prevents memory leaks and ensures that the server does not waste resources on redundant loops. The third constraint is that drawings must render at a steady rate, completing at exactly the 30 second mark, followed by strict 15 second steal windows. This means the source of truth for the timer must reside on the server, not the client's browser. If a player's internet lags, the game must snap to the current state rather than slowing down the clock for everyone else. Lastly the client staff currently lacks the technical expertise to set up or maintain the web-based environment. The implication is the development process must include containerization to package the software with all its dependencies.

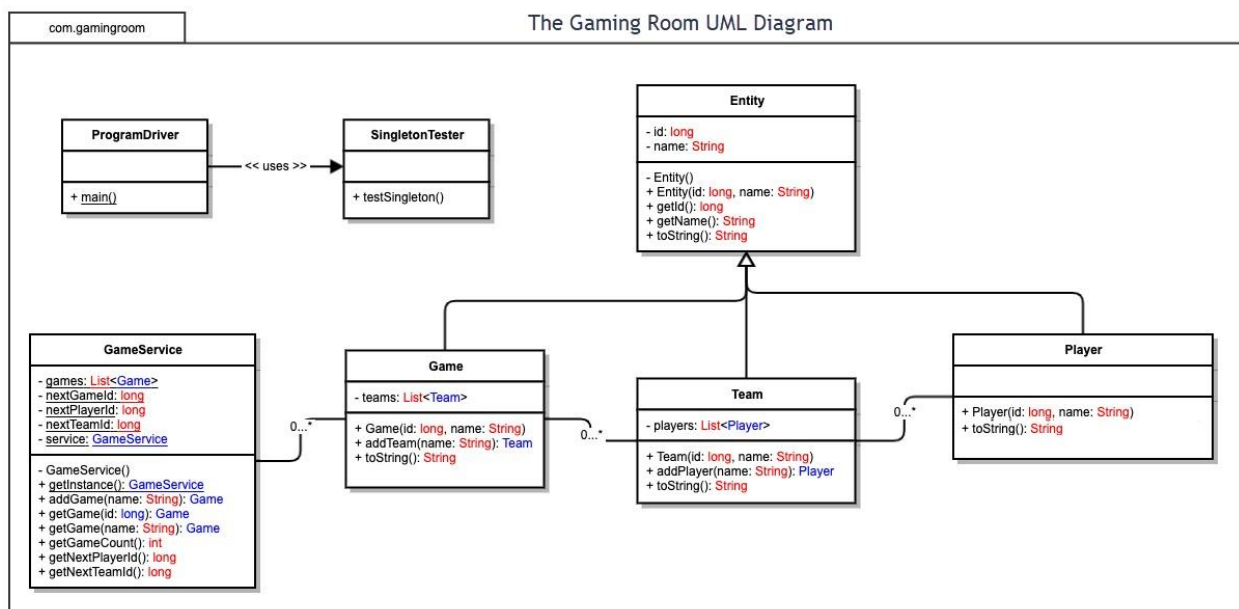
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

In this diagram Inheritance is the primary driver for efficiency. The Entity class acts as a base parent, ensuring that Game, Team, and Player all share a common foundation of a unique id and name. This satisfies the client's requirement for unique identifiers across all objects without redundant code. The relationship is a nested Aggregation: a GameService manages multiple Game objects, which in turn contain Team lists, which finally house Player lists. This creates a logical chain of command for data management.

The design enforces the Singleton Pattern within the GameService class noted by the service: GameService attribute and getInstance() method. This ensures that only one instance of the game manager exists in memory, preventing the conflict of multiple game states running simultaneously. Also Encapsulation is used to protect the integrity of the data; by keeping lists and ID counters private and only accessible through defined methods like addTeam() or getNextPlayerId(), the system prevents external classes from corrupting the game's internal state.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
--------------------------	-----	-------	---------	----------------

Server Side	It is not a standard industry choice for high scale web hosting. Apple discontinued dedicated server hardware years ago, which makes it difficult and expensive to scale to thousands of players in a modern data center or cloud environment.	Linux is the best choice for hosting the server side application. It is the industry standard for web servers because of its high performance, stability, and ability to scale horizontally. Also because it is open source, there are no licensing costs for the OS.	Windows has a strong server environment with powerful integration for .NET applications. But it has high per core licensing costs and usually has a larger resource footprint than Linux. For a web based game, it is a good but more expensive alternative to Linux.	Mobile devices (iOS/Android) cannot act as the server for this application. They can only serve as clients that communicate with the centralized back end server via the web browser.
Client Side	Supporting Mac clients requires testing across Safari and Chrome. Since the application is web based, development time is minimized as the HTML/CSS/JS code is mostly shared. The primary cost is hardware for testing and ensuring the design handles Retina displays correctly.	Linux desktop users are easily supported through web browsers like Firefox. There is not too much work because standard web technologies used for Windows and Mac will typically work without additional modification.	Because Windows is a primary desktop platform it requires extensive browser compatibility testing (Chrome, Edge). Development focus must ensure the responsive interface handles various window sizes and input methods	To support both iOS and Android Responsive Web Design (RWD) is needed. This increases initial development time and expertise requirements, because the team must optimize the UI for small touchscreens and manage specific notched displays found on modern mobile devices.

Development Tools	Java (JDK) is the primary language for implementing the Singleton Pattern and Domain Model. IDEs like IntelliJ IDEA or Eclipse are used on Mac, with Safari Web Inspector for debugging WebSocket traffic.	Developers on Linux usually use Java with the VS Code or Eclipse IDEs. Linux is also the ideal environment for using Docker CLI to build the containers required to package the software for the client.	Visual Studio and VS Code are the main IDEs. Windows supports an array of development tools, but the team may need to use WSL (Windows Subsystem for Linux) to ensure the software built on Windows behaves correctly when deployed to a Linux production server.	To deploy the web app as a native experience on mobile, tools like Capacitor or Cordova are used to wrap the HTML interface. For iOS a Mac and a paid Apple Developer Program membership (\$99/year) is required, which is an added licensing cost.
--------------------------	---	---	--	--

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** <Recommend an appropriate operating platform that will allow The Gaming Room to expand Draw It or Lose It to other computing environments.>

Linux is what I would recommend as it provides the necessary performance, stability, and scalability required for a real time, multi user gaming environment. Linux is the industry standard for high scale web hosting and its lightweight resource footprint ensures the server can handle complex WebSocket communication without unnecessary latency. Also with containerization technologies like Docker on a Linux host, the team can package the entire application with its dependencies, which addresses the requirement for ease of maintenance by non technical staff. Linux will also ensure that the game engine will remain consistent when running on a local development machine or a cloud production server. It also provides the easiest pathway for horizontal scaling as your user base grows. Lastly Linux will lessen licensing costs while maximizing the robustness of the server side architecture so that the game is the source of truth for all connected players.

2. **Operating Systems Architectures:** <Describe the details of the chosen operating platform architectures.>

The kernel space will manage core hardware and system resources, and the user space, which runs applications and services. This design will allow Linux to handle memory, process scheduling, and networking stacks while dynamically loading only necessary drivers. Linux uses kernel level primitives like namespaces and cgroups so the essential isolation needed for containerization technologies like Docker will be provided. This architecture ensures that application environments are highly portable, allowing developers to package code with its dependencies for consistent execution across different computing environments. This

lightweight footprint will allow Linux to maximize available hardware resources for high concurrency tasks like real time WebSocket communication.

3. **Storage Management:** <Identify an appropriate storage management system to be used with the recommended operating platform.>

My recommendation for a storage management system would be Docker Volumes to pair with the Linux based host. Docker volumes ensure that critical game data persists even if the application containers are stopped, updated, or replaced. It does this by abstracting the storage layer from the container's lifecycle. This allows the client to mount host directories directly into the containers, providing the high speed, reliable I/O performance required for real time game state management. Also, this system integrates well with standard Linux filesystem tools, enabling the client's staff to easily perform backups or monitor data usage without needing deep expertise in container internals.

4. **Memory Management:** <Explain how the recommended operating platform uses memory management techniques for the Draw It or Lose It software.>

Linux manages memory through a virtual memory subsystem that decouples the memory addresses used by the game application from the physical RAM addresses. This system loads only the necessary parts of the application into physical memory, optimizing resource usage while allowing the system to run larger processes than the available physical RAM would permit. The kernel uses an LRU caching algorithm to keep active game states and frequently accessed data in high speed RAM, while moving inactive processes to swap space to maintain system responsiveness. With these memory management techniques the Draw It or Lose It game engine remains isolated and stable even if other background services are running on the server.

5. **Distributed Systems and Networks:** <Knowing that the client would like Draw It or Lose It to communicate between various platforms, explain how this may be accomplished with distributed software and the network that connects the devices. Consider the dependencies between the components within the distributed systems and networks (connectivity, outages, and so on).>

To accomplish this, a central server using the WebSocket protocol is necessary. By offloading game logic and state synchronization to a centralized Linux based server, the system ensures a source of truth that is independent from client side environments. The server's job is to handle heavier things like timer management, score calculation, and message routing. This architecture creates a dependency between the stability of the game and the server's ability to handle real time connections, error handling, etc.

6. **Security:** <Security is a must-have for the client. Explain how to protect user information on and between various platforms. Consider the user protection and security capabilities of the recommended operating platform.>

Implementing WebSocket Secure will protect user information by encrypting all data in transit. This means that game traffic will remain private between the client and the server.

Security modules like SELinux and AppArmor will be useful to enforce strict access policies that prevent the game process from interacting with unauthorized system files or ports. To isolate the application environment Linux has containers, which will limit damage if a specific service is compromised. Lastly, Linux has auditing tools to actively monitor for suspicious login patterns or data access anomalies.